FIELD GUIDE / FREE PREVIEW

Insecure Deserialization Explained

Follow the bytes. Understand the reconstruction. Keep authority at the receiver.

01	02	03	04
SOURCE	READER	STATE	EFFECT

VISUAL EBOOK / OBJECT RESTORATION LAB

Haroon Shahid

Expanded edition • November 2026

The receiver owns the decision

Imagine saving an order, sending it to another service and opening it there. The receiver might read a quantity, rebuild an object, or trigger code that is already installed. Those are different levels of authority. This guide shows you how to tell them apart.

Orchard Books is our fictional shop. A buyer may choose two copies; the catalog sets the price. A note can be restored as plain data, or a richer reader can run a restoration hook. Follow both examples to see the difference between an unsafe business decision and behavior during reconstruction.

You can start with no coding experience. Chapters 1–19 follow values and objects from memory to wire. The next parts teach evidence, compare seven runtimes, and turn the same model into defenses and migration checks.

Draw the flow before trying the lab. Predict the first stage that changes, then compare baseline and repair. Keep separate notes for data acceptance, hook behavior, committed effects and anything you did not establish.

01	SOURCE	Who wrote the representation, and who can change or replay it?
02	READER	Which parser or loader runs, and what types and work can it admit?
03	STATE	Which values are restored, and who has authority over each field?
04	EFFECT	Which callbacks or business operations happen before and after checks?

Practice only on systems you own or have explicit authorization and isolation to assess. Use synthetic roles and harmless markers. This is educational material, not a production serialization library, vendor certification or guarantee of professional results.

Contents

I / Build an object from first principles

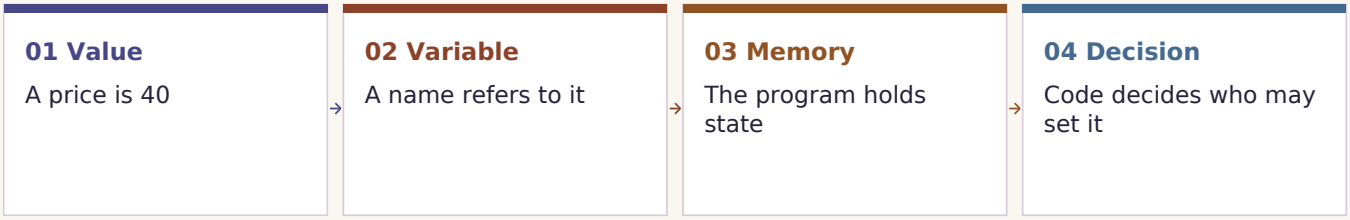
01. A value is not a permission 4

02. Lists, maps and nested records 5

03. Classes, instances and methods 6

This preview contains the introduction and Chapters 1–3. The full book continues through serialization, vulnerability mechanisms, seven runtimes and defensive architecture.

01. A value is not a permission



Imagine a checkout clerk writing a price on a slip. The number 40 is a value. A variable is a name a program uses to refer to a value. Memory is working space the running program uses. None of these things tells you whether the number came from the shop or a customer.

A program executes instructions such as “look up this product” or “compute the total.” A value can affect the outcome without becoming an instruction. If a checkout treats a customer-supplied price as official, a valid number can cause a security failure even if nothing unusual happens during decoding.

Our fictional shop, Orchard Books, will carry the same order through this book. Keep three questions separate: Is the data readable? Is its meaning allowed? Who has the authority to set it? A decoder can help with the first question. It cannot answer the other two on its own.

CHECK YOUR UNDERSTANDING

1.1 What is the difference between a value and a variable?

1.2 Can a perfectly valid number be unsafe to trust?

02. Lists, maps and nested records

01 List
Items in order

02 Map
Names paired with values

03 Record
Fields with meaning

04 Nested state
Records can contain records

A list keeps items in order. A map associates keys with values. A record groups fields that have meaning together. In an order, items may be a list of records; each item has a product identifier and quantity. The full order can also hold an address record.

These shapes are useful because they describe a request precisely. They also create more places for unwanted data. If the receiver checks only the top-level fields, a nested item might still contain a caller-supplied unit price. If a map accepts duplicate names, two components may use different values.

Draw the data on paper before choosing a format. Circle the fields the customer is allowed to provide, such as quantity. Mark the fields Orchard must derive, such as the actual unit price. Then set limits on nesting, list size and string length. Shape and ownership are separate constraints.

CHECK YOUR UNDERSTANDING

2.1 Why can checking only the outer object fail?

2.2 Does a field being present in the schema mean the caller may set it?

03. Classes, instances and methods

01 Class
Defines a kind of object

02 Instance
One object of that kind

03 State
Fields on the instance

04 Method
Behavior that uses the state

A class describes a kind of object. An instance is one actual object created from that class. Imagine an order form as a design and two filled forms as separate instances. In code, a class can also define methods: named behaviors that read or change object state.

Orchard can have an Order with a quantity of two and a method that asks the catalog for a total. Another Order has quantity five. The class's behavior can be shared even though the two instances hold different state. A serialized record generally contains a representation of selected state. A runtime can reconnect that data to existing class behavior during reconstruction.

This distinction explains why the sender need not send method source code for a dangerous effect. If the receiver selects a class that already has a hook, it can run local code. Whether this occurs depends on the reader, registered types and lifecycle rules. A plain JSON record by itself does not run a method.

CHECK YOUR UNDERSTANDING

3.1 Do two instances of one class have to share field values?

3.2 Where can behavior come from if the message holds only data?

Try the complete method

The complete edition includes 65 illustrated chapters, 155 explained checks, 14 worked traces and 28 guided local scenarios. The workbook adds 20 fictional review cases and reusable evidence templates.

Start with the healthy order. Then compare an unauthorized price and a hook that runs before rejection. The capstone asks you to repair those failures independently and verify the delayed worker path.

Published by Haroon Shahid. Prepared with AI assistance. Individual learning license; no resale or redistribution of the paid publication. The unchanged free preview may be shared with attribution. The optional lab is an original teaching implementation. For help or corrections, reply to your Gumroad receipt or email gaknt2@gmail.com.